

STAMPT · SECURITY REVIEW

Stampt Escrow Review.

A security review of Escrow Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Stampt	Escrow Review	Guardian	February 23 – March 25, 2026	April 1, 2026

§ 00

Contents.

27 pages — overview, confidence ranking, scope, 16 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 13 findings	P. 09
	Remediation Review · 3 findings	P. 23
§ 05	Disclaimer	P. 26
§ 06	About Guardian	P. 27

§ 01

Overview.

Guardian reviewed Escrow Review across 2 review rounds. The final report includes 1 Medium, 4 Low, 11 Informational findings.

AUDIT FIRM

Guardian

CLIENT

Stampt

PROJECT

Escrow Review

PREPARED BY

Guardian

AUDIT WINDOW

February 23 – March 25, 2026

FINAL REPORT

April 1, 2026

METHODOLOGY

Smart contract · Manual · Main Review ·
Remediation Review

1.1 Confidence Ranking

5 / 5

Very High
Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.



1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	0	0	0	0	0
Medium	1	0	0	0	1
Low	4	0	2	0	2
Informational	11	0	11	0	0
All severities	16	0	13	0	3

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Escrow Review.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
M-01	Rent Payer Can Be Tricked For Escrow Payment	Validation	MEDIUM	RESOLVED	p. 9
L-01	Escrow Created Before Config Initialization	Validation	LOW	ACKNOWLEDGED	p. 11
L-02	Escrow Amount Not Enforced On Payout	Logical Error	LOW	ACKNOWLEDGED	p. 12
L-03	No Minimum Amount Check Can Cause Griefing	Validation	LOW	RESOLVED	p. 13
L-04	PermanentDelegate token Can Drain Escrow Vault	Trust Assumptions	LOW	RESOLVED	p. 14
I-01	Rent Is Routed To Treasury In Cancel/Refund	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 15
I-02	Transfer Fee Mints Break Escrow Resolution	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 16
I-03	Refund Blocked If Initializer ATA Closed	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 17
I-04	No Duplicate Check On Treasury and Admin Update	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 18
I-05	Incorrect Error Codes in Migrate Escrow	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 19
I-06	No Event Emitted for Escrow Migration	Events	INFORMATIONAL	ACKNOWLEDGED	p. 20
I-07	Refunds Can Be Redirected To Any New Address	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 21
I-08	Gift Key Can Be Unusable	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 22

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
REMEDIATION REVIEW					
I-01	Warning About Fix on M-01	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 23
I-02	High Decimal Mint DoS	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 24
I-03	Unfixed I-02	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 25

MEDIUM **M-01**

RESOLVED

Rent Payer Can Be Tricked For Escrow Payment

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/contracts-team1-1771428637598/blob/946e3dc143d8c6a46c5ff5460e5f68a597e02173/programs/gift-escrow/src/lib.rs#L622>

REVIEW Main Review

DESCRIPTION

In this protocol, account rent for escrows is subsidized by the protocol via a designated `rent_payer` signer (e.g., a backend or frontend-controlled bot). The `initialize_gift_escrow` instruction requires both `initializer` and `rent_payer` to be signers of the same transaction.

However, if the same keypair is used for both roles, a single signature satisfies both signer constraints. In this case, an attacker can construct a transaction that sets:

- `initializer` = subsidy bot pubkey
- `rent_payer` = subsidy bot pubkey
- `initializer_token_account` = a token account controlled by the bot
- `claim_authority` = attacker-controlled key

If the subsidy bot signs this transaction to cover the rent, it implicitly authorizes a token transfer from its own token account into the escrow vault. The attacker can later claim the escrowed tokens using the chosen `claim_authority`.

When a subsidy bot key is reused as both `rent_payer` and `initializer`, and it holds token balances, an attacker can cause token transfers from the bot's accounts, resulting in direct loss of funds.

RECOMMENDATION

One option is to handle this off-chain by ensuring the `rent_payer` account used for rent subsidization holds only SOL and never holds any SPL tokens or associated token accounts. Additionally, the signing bot/account should enforce that the `initializer` is never the `rent_payer` itself.

Alternatively, if an on-chain solution is desired, consider enforcing `rent_payer != initializer` during escrow creation. However, this would prevent users from creating escrows directly when they also pay the rent themselves, which may be a legitimate use case that should not be blocked. This approach should only be adopted if the expected usage model is that the protocol always subsidizes rent and users never pay rent themselves.

M-01 Rent Payer Can Be Tricked For Escrow Payment

CONTINUED

— RESOLUTION

Status: Resolved.

LOW **L-01**

ACKNOWLEDGED

Escrow Created Before Config Initialization

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/contracts-team1-1771428637598/blob/946e3dc143d8c6a46c5ff5460e5f68a597e02173/programs/gift-escrow/src/lib.rs#L588-L627
REVIEW	Main Review

DESCRIPTION

The `initialize_gift_escrow` instruction does not require the Config PDA to exist as an input account. This means escrows can be created and tokens deposited into vaults before `initialize_config` has been called.

However, all resolution paths: `claim`, `cancel`, and `refund_expired`; require a valid `Config` account with `seeds = [b"config"]`, `bump = config.bump` and a treasury that matches `config.treasury`.

RECOMMENDATION

Consider requiring the `Config` account in `InitializeGiftEscrow` to guarantee that no escrow can be created until protocol configuration is finalized.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Escrow Amount Not Enforced On Payout

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/contracts-team1-1771428637598/blob/946e3dc143d8c6a46c5ff5460e5f68a597e02173/programs/gift-escrow/src/lib.rs#L277-L278
REVIEW	Main Review

— DESCRIPTION

Escrow payments transfer the entire vault balance rather than a predetermined escrow amount. If the recipient expects an exact amount and enforces strict equality checks, the claim may fail.

— RECOMMENDATION

Consider transferring only the intended escrow amount to the recipient, and add handling for any excess balance in cases where the vault receives donations or mistakenly sent funds.

— RESOLUTION

Status: Acknowledged.

LOW **L-03**

RESOLVED

No Minimum Amount Check Can Cause Griefing

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/contracts-team1-1771428637598/blob/946e3dc143d8c6a46c5ff5460e5f68a597e02173/programs/gift-escrow/src/lib.rs#L40>

REVIEW Main Review

— DESCRIPTION

Escrow creation is permissionless and there are no minimum or maximum escrow amount limits. Since the protocol covers the rent payments for these escrows, a malicious user could create a large number of dust escrows, causing the protocol to incur substantial rent costs for escrow contracts and vaults holding tiny amounts. The impact could be amplified by creating non-expiring escrows with negligible amounts, preventing the treasury from ever recovering the rent paid for these escrows.

— RECOMMENDATION

Introduce a minimum escrow amount to mitigate griefing. Even a small threshold (e.g., \$1) could be sufficient to prevent these cases. Additionally, consider enforcing a maximum escrow amount.

— RESOLUTION

Status: Resolved.

LOW **L-04**

RESOLVED

PermanentDelegate token Can Drain Escrow Vault

CATEGORY Trust Assumptions

LOCATION <https://github.com/GuardianOrg/contracts-team1-1771428637598/blob/main/programs/gift-escrow/src/lib.rs>

REVIEW Main Review

DESCRIPTION

Token-2022 mints with the `PermanentDelegate` extension allow a designated address to transfer tokens from any token account of that mint without requiring the account authority's signature. If an escrow is created using such a mint, the permanent delegate can drain the vault directly through the token program, bypassing the escrow PDA authority entirely. This would cause all resolution paths (`claim`, `cancel`, `refund_expired`) to fail since they all enforce `vault_balance >= escrow.amount`. The escrow becomes permanently stuck, and the rent paid for the escrow and vault PDAs is locked with no recovery path.

RECOMMENDATION

Reject mints that have a `PermanentDelegate` configured.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Rent Is Routed To Treasury In Cancel/Refund

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/contracts-team1-1771428637598/blob/946e3dc143d8c6a46c5ff5460e5f68a597e02173/programs/gift-escrow/src/lib.rs#L731-L732
REVIEW	Main Review

— DESCRIPTION

When an escrow is claimed, canceled, or refunded after expiry, both the Escrow PDA and the vault token account are closed. In all cases, the lamports held as rent in these accounts are transferred to the protocol treasury, rather than being refunded to the initializer or claimer.

Ideally, rent should be refunded to the initializer when the escrow gets cancelled or refunded. The protocol team acknowledged that rent is subsidized for users and paid by the protocol itself. However, if users create escrows and pay the rent themselves, those rent payments are transferred to the treasury and are lost during refunds.

— RECOMMENDATION

Document this behavior and inform users of this scenario.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Transfer Fee Mints Break Escrow Resolution

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/contracts-team1-1771428637598/blob/946e3dc143d8c6a46c5ff5460e5f68a597e02173/programs/gift-escrow/src/lib.rs#L27
REVIEW	Main Review

DESCRIPTION

Token-2022 mints with the `TransferFeeConfig` extension can be used to create escrows. When a `TransferFee` mint is used, the vault receives less than the requested amount due to the on-transfer fee deduction.

The stored `escrow.amount` reflects the full pre-fee value, causing the `vault_balance >= escrow.amount` guard to fail on all resolution paths (claim, cancel, refund_expired).

RECOMMENDATION

Consider rejecting mints with the `TransferFeeConfig` extension during `initialize_gift_escrow` before the token transfer occurs.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

ACKNOWLEDGED

Refund Blocked If Initializer ATA Closed

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/contracts-team1-1771428637598/blob/main/programs/gift-escrow/src/lib.rs#L789-L791
REVIEW	Main Review

DESCRIPTION

The `refund_expired` instruction is designed to be permissionless, anyone can call it after expiry to return funds to the initializer. However, it requires the caller to pass a valid `initializer_token_account` that satisfies two manual checks in the instruction body:

```
require!(
  ctx.accounts.initializer_token_account.mint == escrow.mint,
  EscrowError::InvalidTokenAccount
);
require!(
  ctx.accounts.initializer_token_account.owner == escrow.initializer,
  EscrowError::InvalidTokenAccount
);
```

The `initializer_token_account` is declared as a plain `InterfaceAccount<'info, TokenAccount>` with only `#[account(mut)]`. it must already exist on-chain. If the initializer closes their Associated Token Account after creating the escrow (e.g., to reclaim rent, or via wallet cleanup), there is no valid account to pass.

RECOMMENDATION

Consider documenting this behavior so that users are aware of it.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

ACKNOWLEDGED

No Duplicate Check On Treasury and Admin Update

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/contracts-team1-1771428637598/blob/946e3dc143d8c6a46c5ff5460e5f68a597e02173/programs/gift-escrow/src/lib.rs#L152-L165
REVIEW	Main Review

— DESCRIPTION

The `admin` can update the treasury account and admin account. However, there is no check to ensure the new treasury is different than the old treasury, or the new admin is different than the old admin. These functions can be called with the same addresses.

— RECOMMENDATION

Consider having a duplicate check in these functions.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-05

ACKNOWLEDGED

Incorrect Error Codes in Migrate Escrow

CATEGORY Informational

LOCATION <https://github.com/GuardianOrg/contracts-team1-1771428637598/blob/main/programs/gift-escrow/src/lib.rs#L537-L555>, <https://github.com/GuardianOrg/contracts-team1-1771428637598/blob/main/programs/gift-escrow/src/lib.rs#L528-L532>

REVIEW Main Review

DESCRIPTION

In `migrate_escrow`:

1. `EscrowError::InvalidEscrowStatus` is used for discriminator validation and account size validation. These checks verify that the account has the correct Escrow discriminator and matches the expected old-format size; they are format/version checks, not escrow status checks.
2. `EscrowError::InvalidTokenAccount` is used for a program ownership check (`escrow_info.owner == ctx.program_id`). This check validates that the escrow account is owned by the gift-escrow program, not that a token account is valid

RECOMMENDATION

Consider using distinct error variants to clearly communicate the actual failure.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-06**

ACKNOWLEDGED

No Event Emitted for Escrow Migration

CATEGORY	Events
LOCATION	https://github.com/GuardianOrg/contracts-team1-1771428637598/blob/main/programs/gift-escrow/src/lib.rs#L525
REVIEW	Main Review

DESCRIPTION

The `migrate_escrow` instruction does not emit any event upon successful migration of an old-format escrow to the new format. Every other state-changing instruction in the program emits an event:

`initialize_gift_escrow` emits `EscrowInitialized`, `claim` emits `EscrowClaimed`, `cancel` emits `EscrowCanceled`, `refund_expired` emits `EscrowExpiredRefund`, `initialize_config` emits `ConfigInitialized`, `update_treasury` emits `TreasuryUpdated`, and `update_admin` emits `AdminUpdated`.

RECOMMENDATION

Consider emitting a `EscrowMigrated` event.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-07**

ACKNOWLEDGED

Refunds Can Be Redirected To Any New Address

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/contracts-team1-1771428637598/blob/946e3dc143d8c6a46c5ff5460e5f68a597e02173/programs/gift-escrow/src/lib.rs#L461
REVIEW	Main Review

— DESCRIPTION

`refund_expired` is permissionless, and anyone can refund an expired escrow to the initializer. During this action, the only check performed is that the provided token account is owned by the initializer; therefore, no other party can receive the funds.

However, anyone can create token accounts on behalf of any user. As a result, an attacker can create a new token account for the initializer and refund the tokens to that account. While there is no direct incentive to do so, the initializer may not be aware that a new token account has been created on their behalf and may not realize that the refunded funds were sent to a different account.

— RECOMMENDATION

Be aware of this behavior and document it. Alternatively, only allow the initializer to refund the escrow themselves.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-08**

ACKNOWLEDGED

Gift Key Can Be Unusable

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/contracts-team1-1771428637598/blob/946e3dc143d8c6a46c5ff5460e5f68a597e02173/programs/gift-escrow/src/lib.rs#L617
REVIEW	Main Review

DESCRIPTION

During escrow creation, the `claim_authority` (gift key) is not validated against obvious invalid or unusable values unlike the checks in admin and treasury updates. In particular, it may be set to default pubkey or system account etc.

Since claiming the escrow requires the `claim_authority` to sign the transaction, using such a key makes the escrow unclaimable, and must be cancelled/refunded later

RECOMMENDATION

Consider validating gift key against obvious invalid ones similar to other instructions.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Warning About Fix on M-01

CATEGORY	Warning
LOCATION	https://github.com/GuardianOrg/contracts-team1-1771428637598/blob/f984ad1b3c78f23933e22568712a2cf4f20fe610/programs/gift-escrow/src/lib.rs#L105
REVIEW	Remediation Review

— DESCRIPTION

The fix for M-01 ensures on-chain that the initializer is not the rent payer, preventing the bot from being tricked into paying tokens in addition to rent. However, it is important to note that, on the backend, the bot must still ensure it only signs transactions where `bot == rent_payer`

The onchain restriction can be satisfied by setting:

- `initializer` = subsidy bot pubkey
- `rent_payer` = attacker pubkey

An attacker may attempt to trick the bot into paying the tokens while covering the rent themselves. Off-chain checks should ensure this never occurs.

— RECOMMENDATION

Ensure that the rent subsidy bot never signs a transaction where `rent_payer` differs from the bot itself, for the previous fix to be fully effective.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

High Decimal Mint DoS

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/contracts-team1-1771428637598/blob/f984ad1b3c78f23933e22568712a2cf4f20fe610/programs/gift-escrow/src/lib.rs#L62
REVIEW	Remediation Review

DESCRIPTION

`initialize_gift_escrow` now enforces a dust floor `min_amount = 10_u64.checked_pow(decimals.saturating_sub(1) as u32)`. For any mint with >20 decimals the exponentiation overflows u64, and the escrow creation reverts with `AmountBelowMinimum`

While tokens with such high decimal precision are extremely rare on Solana, they may still exist and cannot be supported by the gift escrow.

RECOMMENDATION

Document this behavior.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

ACKNOWLEDGED

Unfixed I-02

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/contracts-team1-1771428637598/blob/f984ad1b3c78f23933e22568712a2cf4f20fe610/programs/gift-escrow/src/lib.rs#L256
REVIEW	Remediation Review

— DESCRIPTION

The `SECURITY_HARDENING.md` file states that Token-2022 with `TransferFee` is supported. However, as noted in the previous I-02 issue, the program cannot support these tokens.

— RECOMMENDATION

Update the documentation if these tokens are not supported and inform users accordingly. Otherwise, update the program to support these tokens.

— RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.